

Genesis Engine: A Falsifiable, Deterministic Laboratory for Emergent Civilizations

Author: Micka Delcato **Affiliation:** Independent researcher — Genesis Engine project
Correspondence: github.com/Micka420-collab/genesis-engine **Version:** Preprint v1 — 2026-05-28
License: AGPL-3.0-only (text); CC-BY-4.0 (this document) **Languages:** English (this file) · [Français](#)

Abstract

We present **Genesis Engine**, an open-source artificial-life (alife) laboratory whose design goal is the *observation of emergent civilization-scale phenomena*—language, tools, trade, currency, and collapse—under a strict methodological constraint: **only the laws of physics are hard-coded; everything cultural must emerge from agents, and every emergent claim must be falsifiable**. The system couples a Rust substrate (a deterministic, content-addressed voxel world with Köppen climate, geology, and hydrology) to a Python simulation layer (genome, metabolism, perception, neuro-evolution, and social dynamics). Reproducibility is enforced by a single pseudo-random function (PRF) source of entropy and by SHA-256 *state fingerprints* recorded in per-run provenance manifests. We formalize four engine-level invariants (I-1...I-4) and a Popperian *falsifiability ledger* that separates **verified observables** from **open hypotheses**. We report an empirical demonstration of run-level determinism (identical seed → identical fingerprint; distinct seed → distinct fingerprint), a verified genetic observable (Wright’s inbreeding coefficient $F = 0.2500$ for full siblings), and a deterministic epidemic observer. We are explicit about what is **not** yet proven: endogenous currency, Tainter-style collapse, sexual selection, epistemic bubbles, and bidirectional niche construction remain *hypotheses to be tested*, not results. The contribution is less a single result than a **methodology**: an apparatus in which claims about emergence can be pre-registered, reproduced bit-for-bit, and refuted.

Keywords: artificial life, emergence, determinism, reproducibility, falsifiability, multi-agent simulation, cultural evolution, complex systems.

1. Vision and theory

1.1 The thesis

Most simulations of “civilization” *script* the outcome: the rules already contain the answer (tech trees, fixed recipes, “salt is money”). Genesis Engine adopts the opposite stance, which we call **ZERO PRE-SCRIPT**:

If a phenomenon is interesting because it emerged, then it must not be in the rules. Only physics is given. Language, tools, money, social structure, and collapse must arise—or fail to arise—from the interaction of embodied agents with a physical world.

The intellectual lineage is the artificial-life tradition (Tierra, Avida, Lenia, Polyworld) extended to the *civilizational* scale, and the philosophy of science of Karl Popper: a claim that cannot be refuted is not a scientific claim. The project’s working hypothesis is therefore not “civilization will emerge,” but the

stronger and testable: “**under physics P, agent architecture A, and seed s, observable O crosses threshold θ on ≥ 3 independent seeds.**” Anything weaker is marked as an open question.

1.2 Why determinism is load-bearing

Emergence claims are notoriously fragile: a result that appears once, on one machine, under one random draw, is anecdote, not evidence. Genesis Engine makes determinism a *first-class invariant*. Given the same (seed, configuration, region), the world and its trajectory must be reproducible bit-for-bit, and the resulting state must hash to an identical SHA-256 fingerprint. This converts “I saw money emerge” into “money emerged in run c2e03804..., reproducible by anyone on the same commit.” Determinism is the spine on which falsifiability hangs.

1.3 From the origin of life upward

The long-term ambition is a stack that begins at the substrate (protocells → photosynthesis → oxygenation → fauna) and lets sapient agents appear without scripted founders—“free will as the origin,” in the project’s own phrasing. This document does **not** claim that full arc has been demonstrated. It documents the *apparatus* and the *method*, and reports the subset of behaviour that is currently backed by a passing, deterministic test.

2. Background and related work

Tradition	Example systems	What Genesis Engine borrows	What it changes
Digital evolution	Tierra, Avida	Self-replication, selection on a substrate	Adds an Earth-anchored physical world and embodiment
Continuous alife	Lenia, particle life	Emergence from local rules	Adds discrete agents with genomes and cognition
Embodied agents	Polyworld, neural agents	Neuro-evolution (NEAT-style plasticity)	Couples to climate/geology/hydrology at planetary scale
Agent-based social science	Sugarscape, NetLogo models	Trade, resource gradients	Forbids scripted institutions; they must emerge
Reproducible ML/science	Pre-registration, model cards	Provenance + pre-registration of hypotheses	Per-run SHA-256 state fingerprints, Popperian ledger

The distinguishing commitment is methodological: not *that* phenomena emerge, but that **each emergence claim carries its own refutation condition and a reproducible hash.**

3. Methodology

3.1 Single source of entropy (PRF)

All stochasticity flows through one pseudo-random function, `engine.core.prf_rng`. Use of unseeded `random.*` is prohibited by contribution rules and review. This guarantees that (seed, config) fully

determines a run.

3.2 Engine-level invariants

Four invariants form the substrate’s contract. They are intended to be guarded by tests; §6.4 reports honestly on the current enforcement gap for three of them.

ID	Invariant
I-1	Same (seed, config) → bit-identical chunks (single- and multi-threaded)
I-2	NaN-safe, cross-platform hashing in every content-addressable structure
I-3	Concurrent coalescing: N parallel callers for one coordinate trigger exactly one generate() (panic-safe)
I-4	Mutated chunks survive cache eviction and snapshot/restore

3.3 State fingerprint and provenance

Each long run is wrapped in experimental_run, which writes a manifest.json capturing: git commit, SHA-256 of pyproject.toml, Python version, platform, timing, the run summary, and a **state fingerprint** = SHA-256 over the *deterministic simulation state*. Crucially, the fingerprint **excludes** volatile host/timing fields (throughput, wall-clock, timestamps, absolute paths); §6.1 reports a defect we found and fixed in exactly this area. A manifest is written even on crash, so partial runs remain diagnosable.

3.4 Pre-registration and the falsifiability ledger

Before a hypothesis-driven run, the experimenter copies PREREGISTRATION_TEMPLATE.md, states the hypothesis, the quantitative prediction, and the stopping/refutation condition, and **commits it before running** (the git graph is the audit trail). Results are entered in FALSIFIABILITY.md with one of five statuses: confirmed / pending / null / refuted / superseded. **Confirmation requires the observable to cross its threshold on ≥ 3 distinct seeds at the same commit.** No public claim may be cited without its ledger entry.

3.5 Wave discipline

Features ship as numbered “Waves,” each accompanied by a smoke script (pnn_smoke.py) and integrated into the main tick. This keeps the claim surface aligned with the test surface: a Wave without a green smoke is not “done.”

4. System architecture

L4 Civilization

trade · construction · polity · language · observers

L3 Cognition

local perception · NEAT-style plasticity · action selection

L2 Biology

256-D genome · metabolism · anatomy · sexual selection

L1 World

Genesis · Köppen climate · biomes · resources

L0 Physics

thermodynamics · gravity · hydrology · erosion

▲

Rust substrate (native/world-engine): WorldGraph, streaming,
GPU, snapshot/restore – bridged to Python via PyO3 (genesis_world)

- **Rust substrate** (`native/world-engine/` , 24 crates): a content-addressed, chunked voxel world with deterministic generation, Köppen classification, geology, hydrology, weather, meshing, and snapshot/restore. Exposed to Python through a PyO3 wheel.
- **Python runtime** (`runtime/engine/`): the agent and society layers—genome, metabolism, perception, cognition, communication, construction, trade, and a suite of *observers* (epidemic, lineage, vision) that measure rather than drive the simulation.
- **Bridge** (`engine.rust_bridge`): selects the native backend when available and validated, otherwise a Genesis-anchored Python mock. §6.2 reports a real bridge defect we fixed.

5. Reproducibility and verified observables

The following are backed by tests that pass deterministically. We label each with its guardian so a reader can re-run it.

5.1 Run-level determinism (empirical I-1)

Using the project’s own pipeline, we ran a 300-tick civilization scenario three times:

```
python runtime/scripts/civilization_pipeline.py \
  --experiment paper --seed <S> --ticks 300 --founders 12
```

Run	Seed	State fingerprint (SHA-256, abbreviated)
A	0xC1A71CE0	c2e038049950056e105503ff8430281617edab3a...
B	0xC1A71CE0	c2e038049950056e105503ff8430281617edab3a...
C	0xBADC0FFEE	7fe0b90867d29608960c4ea2abe939413701c4e4...

Result: A = B (same seed → identical fingerprint) and A ≠ C (distinct seed → distinct fingerprint). The associated `world_signature` for the baseline is `7a6d7eb0bc1c8140205c772d2fc3935b66bd0ba9e4dac9647f997910b4b68304` . Provenance recorded: Python 3.14.3, Windows-10, `pyproject.toml` SHA-256 `6c7e2555...` . This is a direct, reproducible demonstration of the determinism invariant at the pipeline level.

5.2 Genetic structure — Wright’s inbreeding coefficient

Guardian: `p71_lineage_observer_smoke` (9/9 PASS). For full siblings the observer computes **F = 0.2500** exactly, and **F = 0.0000** for unrelated pairs—the textbook values. Because the lineage observer *measures* the genome graph rather than imposing a result, this is evidence that the inheritance substrate is correct, not that a social outcome was scripted.

5.3 Epidemic dynamics — deterministic observer

Guardian: `p70_epidemic_observer_smoke` (9/9 PASS), including an explicit *inter-run determinism* check over the contact-graph snapshots. The observer tracks an SIR-style state per pathogen on an emergent contact network. We report the machinery and its determinism as verified; specific basin numbers (e.g. an *R*₀ figure) depend on the chosen pathogen scenario and should be cited per-scenario, not as a universal constant.

5.4 Substrate breadth (smoke-backed)

The validated pipeline installs and ticks, deterministically, the modules: `climate`, `genesis`, `geology`, `hydrology`, `marine`, `meteorology`, `wildfire`, plus a multi-rate coupler and the observer suite. 173+ Python tests and a battery of `pNN` smokes back the substrate; see the repository's `NEXT-SPRINT.md` and `FALSIFIABILITY.md` for the live count.

6. What we fixed to make the apparatus scientifically valid

In preparing this paper we ran the apparatus adversarially and corrected three defects that bear directly on scientific validity.

6.1 The state fingerprint was not reproducible

The citable state fingerprint hashed the **entire** run payload, including wall-clock throughput (`tps`), wall-clock seconds, and an **absolute** `manifest_path`. Two identical-seed runs therefore produced *different* fingerprints—silently defeating the project's central reproducibility promise. We made `compute_state_fingerprint` strip volatile host/timing keys recursively before hashing, so the fingerprint reflects only deterministic simulation state and is stable **across machines**. §5.1's A = B result is the post-fix verification; regression tests pin the behaviour.

6.2 A wheel-name collision could silently break the native bridge

Two distinct Rust crates compiled to a Python module with the *same* name (`genesis_world`) but *incompatible* APIs. A stale legacy wheel could shadow the canonical one in `site-packages`, crashing the bridge with a cryptic `TypeError`. We hardened the bridge to verify the canonical `PyWorld` API contract before trusting a module as “native,” falling back to the Genesis-anchored mock with an actionable warning otherwise. This converts a silent corruption into an honest, diagnosable state.

6.3 Honest reporting of the execution backend

Run manifests now record `rust_bridge: {native: false, module: MockPyWorld}` when the native wheel is absent, rather than over-claiming native execution. Scientific validity requires the apparatus to report *what it actually did*.

6.4 Threats to validity (open, disclosed)

- **Invariant enforcement gap.** Three of four engine invariants (I-1, I-3, I-4) live in Rust crates whose CI step is currently marked `continue-on-error`, i.e. failures are not yet blocking. The public claim that all four are “guarded by blocking CI” is therefore stronger than reality until those crates compile cleanly and the flag is removed. (I-2 is genuinely blocking.) This is disclosed, not hidden.
- **Single-machine fingerprints.** §5.1 demonstrates determinism on one platform (Windows, Python 3.14.3). Cross-platform bit-identity is an invariant *goal* (I-1/I-2) but is not demonstrated here.
- **Mock vs native.** §5.1 ran against the Python mock backend (the native wheel was absent/legacy on the test host). The mock is Genesis-anchored and deterministic, but it is not the Rust substrate; native-backed reproduction is future work.

- **Earth-realism is partial.** The project’s self-assessed Earth-realism score is ~76% (geology being the weakest dimension). “Realistic” is a direction, not a finished claim.
- **Python version.** Results were produced on Python 3.14, which is *outside* the declared supported range ($\geq 3.11, < 3.14$); the suite nonetheless passes.

7. Open hypotheses (explicitly not proven)

These are the phenomena the apparatus is built to *test*. None may be cited as a result until a pre-registered run validates it on ≥ 3 seeds in `FALSIFIABILITY.md`.

1. **Endogenous currency.** A commodity becomes a medium of exchange purely from observed trades—no hard-coded “money.”
2. **Tainter-style collapse.** Diminishing marginal returns on complexity lead agents to refuse to build/invent and revert to subsistence.
3. **Sexual selection.** Courtship plus a `mate_preference` derived from personality traits yields measurable parent–offspring correlation on visible traits.
4. **Epistemic bubbles.** Communication with a veracity parameter and consistency checks yields emergent trust clusters; persistent liars become isolated.
5. **Bidirectional niche construction.** Agent activity (fire, agriculture, urbanization) alters the substrate, which in turn exerts selective pressure on agents.

8. How to reproduce

```
git clone https://github.com/Micka420-collab/genesis-engine.git
cd genesis-engine
python -m venv .venv && . .venv/bin/activate      # Windows: .venv\Scripts\activate
python -m pip install -e ".[dev]"
make test-python                                # Python test suite
# Reproduce §5.1 determinism:
PYTHONPATH=runtime python runtime/scripts/civilization_pipeline.py \
  --experiment repro --seed 0xC1A71CE0 --ticks 300 --founders 12
# → runtime/experiments/repro_UTC>/manifest.json (compare state_fingerprint)
```

To register a new hypothesis, copy `runtime/experiments/PREREGISTRATION_TEMPLATE.md`, fill it in, commit it **before** the run, then record the outcome in `FALSIFIABILITY.md`.

9. Conclusion and call for collaboration

Genesis Engine’s contribution is an *epistemic apparatus*: a deterministic, Earth-anchored world in which civilization-scale emergence can be pre-registered, reproduced bit-for-bit, and refuted. We have demonstrated run-level determinism, a correct genetic substrate, and a deterministic epidemic observer; we have been explicit that the headline phenomena (currency, collapse, sexual selection, epistemic bubbles, niche construction) remain **hypotheses**, and we have disclosed the apparatus’s current limitations rather than papering over them. We invite alife researchers, Rust/Python engineers,

geographers, and philosophers of science to pre-register hypotheses, attempt refutations, and reproduce or break the fingerprints reported here. *A claim that cannot be refuted is not a result—so come and try to refute these.*

References (selected, indicative)

1. K. R. Popper, *The Logic of Scientific Discovery*, 1959.
2. T. S. Ray, “An approach to the synthesis of life” (Tierra), *Artificial Life II*, 1991.
3. C. Ofria & C. O. Wilke, “Avida: A software platform for research in computational evolutionary biology,” *Artificial Life*, 2004.
4. B. W. Chan, “Lenia — Biology of Artificial Life,” *Complex Systems*, 2019.
5. L. Yaeger, “Computational genetics, physiology, metabolism... (PolyWorld),” *Artificial Life III*, 1994.
6. K. O. Stanley & R. Miikkulainen, “Evolving Neural Networks through Augmenting Topologies (NEAT),” *Evolutionary Computation*, 2002.
7. J. H. Epstein & R. Axtell, *Growing Artificial Societies* (Sugarscape), 1996.
8. J. A. Tainter, *The Collapse of Complex Societies*, 1988.
9. S. Wright, “Coefficients of inbreeding and relationship,” *The American Naturalist*, 1922.
10. W. Köppen, “Das geographische System der Klimate,” 1936.

This document is part of the Genesis Engine repository and versioned alongside the code it describes. Fingerprints and counts are valid as of commit referenced in the repository at publication; re-run the commands in §8 to verify against the current tree.